

1.

The SWISH interface displays a Prolog program in the left pane and its execution in the right pane. The program defines a list of pets and their owners. The right pane shows the query `mascotas(carlos, M).` being executed, with the result `M = []` displayed. The background features a large owl illustration.

```
3 perro(max).
4
5 gato(misu).
6 gato(luna).
7 gato(chanel).
8 gato(orion).
9
10 ave(piolin).
11
12 dueno(ana, firulais).
13 dueno(ana, misu).
14
15 dueno(luis, luna).
16 dueno(luis, orion).
17 dueno(luis, firulais).
18
19 dueno(maria, piolin).
20
21 dueno(julia, chanel).
22
23 dueno(pedro, bruno).
24 mascotas(Persona, Mascotas) :-
25     findall(Mascota, dueno(Persona, Mascota), Mascotas).
```

Query: `mascotas(carlos, M).`
Result: `M = []`

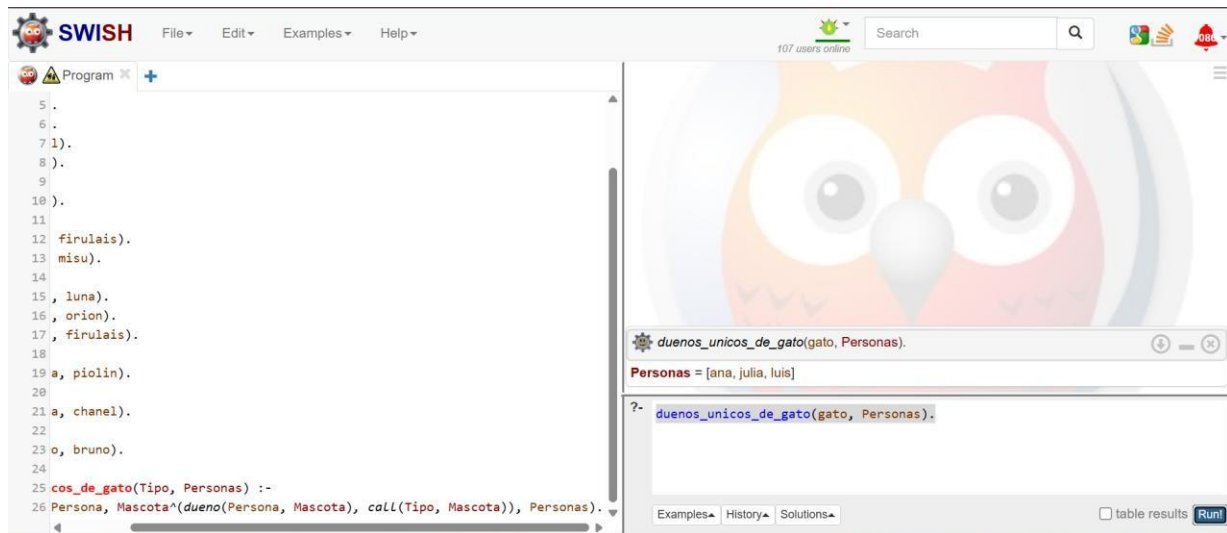
2.

The SWISH interface displays a Prolog program in the left pane and its execution in the right pane. The program defines a list of pets and their owners. The right pane shows the query `duenos_de_gato(Personas).` being executed, with the result `Personas = [ana, julia, luis]` displayed. The background features a large owl illustration.

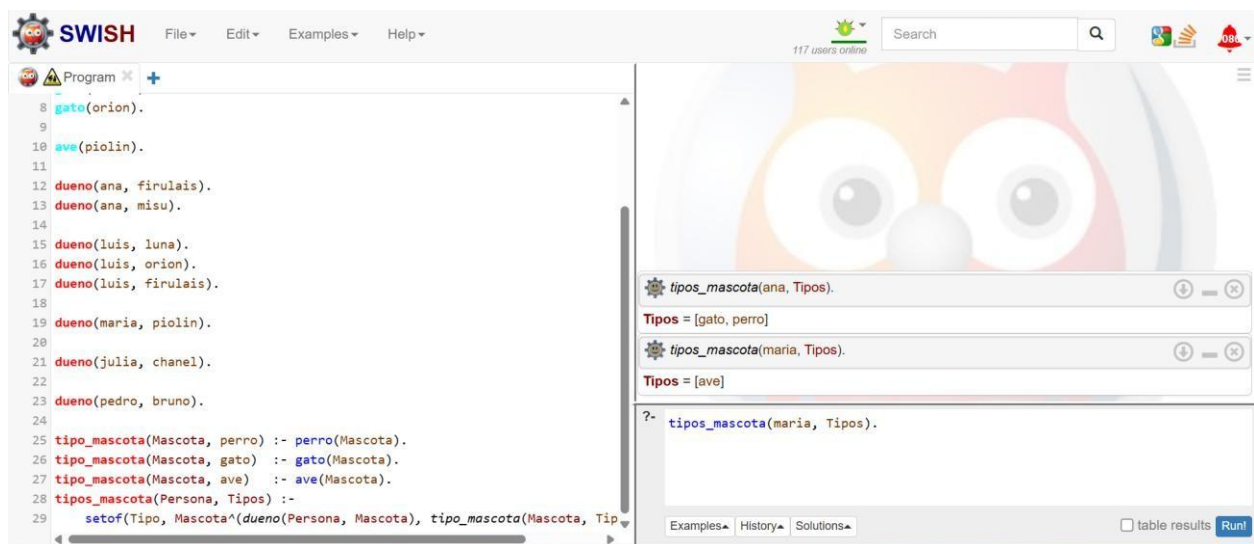
```
4
5 gato(misu).
6 gato(luna).
7 gato(chanel).
8 gato(orion).
9
10 ave(piolin).
11
12 dueno(ana, firulais).
13 dueno(ana, misu).
14
15 dueno(luis, luna).
16 dueno(luis, orion).
17 dueno(luis, firulais).
18
19 dueno(maria, piolin).
20
21 dueno(julia, chanel).
22
23 dueno(pedro, bruno).
24 duenos_de_gato(Personas) :-
25     findall(Persona, (dueno(Persona, Mascota), gato(Mascota)), Lista),
26     sort(Lista, Personas).
```

Query: `duenos_de_gato(Personas).`
Result: `Personas = [ana, julia, luis]`

3.



4.



5.

1. ¿Por qué bagof/3 genera varias soluciones?

Porque en el objetivo hay una variable, Persona, que no aparece en la plantilla. Entonces bagof no junta todo en una sola lista, sino que va generando una lista distinta por cada persona, y toca usar el punto y coma para ver la siguiente.

2. ¿Qué variable está siendo utilizada para realizar la agrupación?

La variable Persona, porque es la que queda libre en la consulta y no está en la plantilla, entonces Prolog la usa para separar los resultados por grupos.

3. ¿Qué ocurriría si se utilizara findall/3?

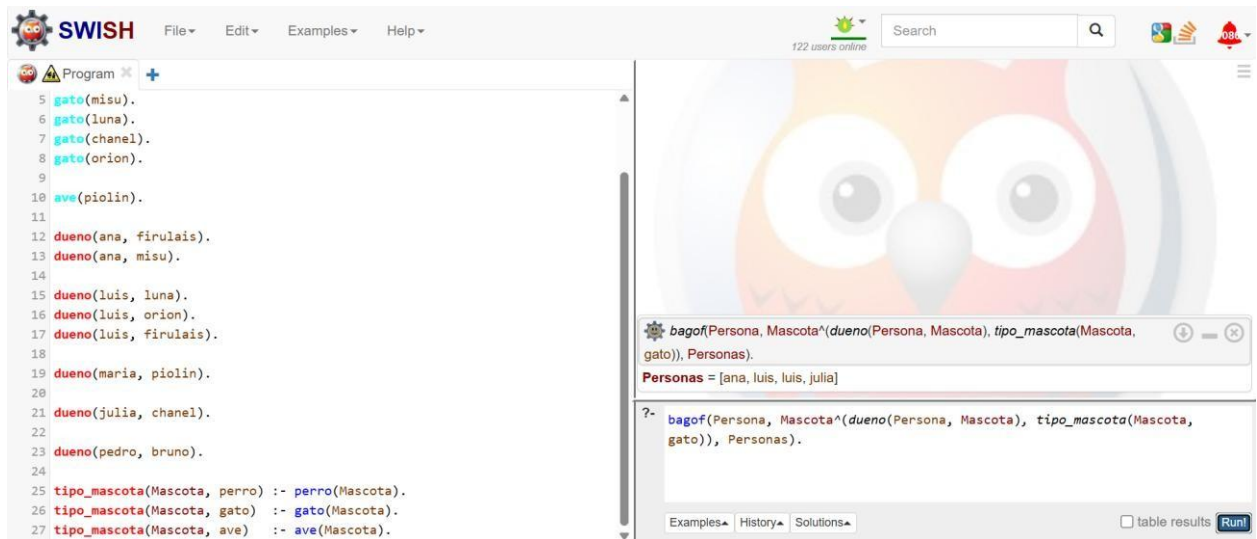
Con findall todas las mascotas quedarían mezcladas en una sola lista, sin importar de quién son, porque findall no agrupa por Persona, simplemente junta todas las soluciones que encuentra.

4. ¿Qué ocurriría si se utilizara setof/3?

Pasaría casi lo mismo que con bagof, o sea que también agrupa por Persona y da una solución a la vez, pero la diferencia es que setof ordena tanto las mascotas dentro de cada grupo como el orden en que van saliendo las personas.

6.

1)



The screenshot shows the SWISH Prolog IDE. On the left, the 'Program' window contains the following code:

```
5 gato(misu).
6 gato(luna).
7 gato(chanel).
8 gato(orion).
9
10 ave(piolin).
11
12 dueno(ana, firulais).
13 dueno(ana, misu).
14
15 dueno(luis, luna).
16 dueno(luis, orion).
17 dueno(luis, firulais).
18
19 dueno(maria, piolin).
20
21 dueno(julia, chanel).
22
23 dueno(pedro, bruno).
24
25 tipo_mascota(Mascota, perro) :- perro(Mascota).
26 tipo_mascota(Mascota, gato) :- gato(Mascota).
27 tipo_mascota(Mascota, ave) :- ave(Mascota).
```

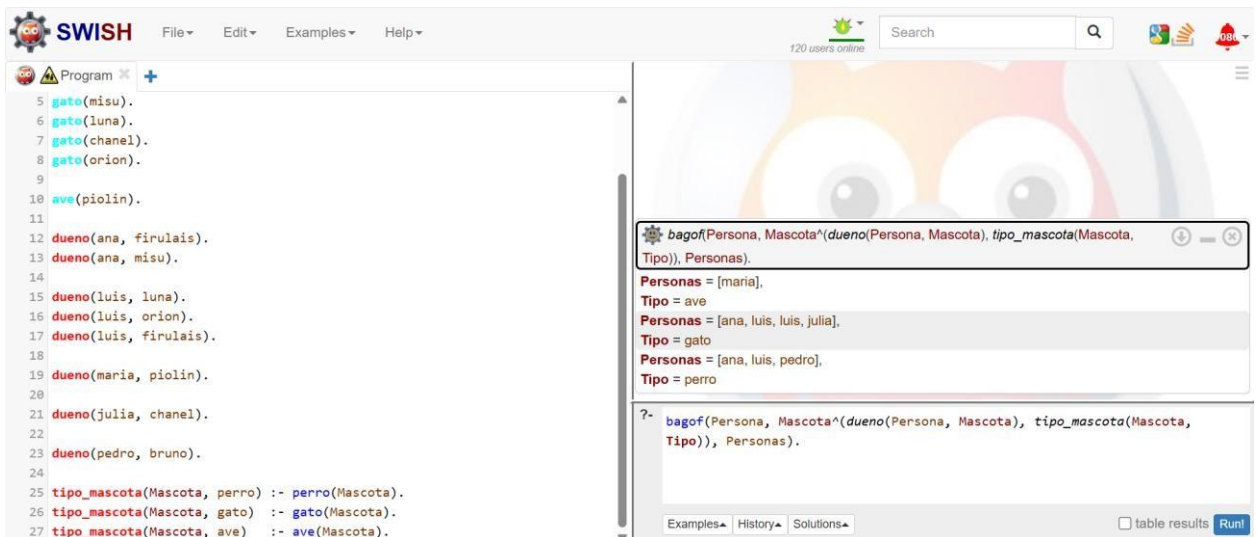
On the right, the 'bagof' query is executed:

```
bagof(Persona, Mascota^(dueno(Persona, Mascota), tipo_mascota(Mascota, gato)), Personas).
```

The result is:

```
Personas = [ana, luis, luis, julia]
```

2)



The screenshot shows the SWISH Prolog IDE. On the left, the 'Program' window contains the same code as in the previous screenshot.

On the right, the 'bagof' query is executed with an additional constraint:

```
bagof(Persona, Mascota^(dueno(Persona, Mascota), tipo_mascota(Mascota, Tipo)), Personas).
```

The result is:

```
Personas = [maria].
Tipo = ave
Personas = [ana, luis, luis, julia].
Tipo = gato
Personas = [ana, luis, pedro].
Tipo = perro
```

7.



File

Edit

Examples

Help

123 users online

Search

Q





Program

+

```

7 gato(chanel).
8 gato(orion).
9
10 ave(piolin).
11
12 dueno(ana, firulais).
13 dueno(ana, misu).
14
15 dueno(luis, luna).
16 dueno(luis, orion).
17 dueno(luis, firulais).
18
19 dueno(maria, piolin).
20
21 dueno(julia, chanel).
22
23 dueno(pedro, bruno).
24
25 persona(Persona) :-
26     dueno(Persona, _).
27
28 todos_tienen_mascota :-
29     forall(persona(Persona), dueno(Persona, _)).

```

todos_tienen_mascota.

true

1

?- todos_tienen_mascota.

Examples

History

Solutions

☐ table results

Run

8.



File

Edit

Examples

Help

120 users online

Search

Q





Program

+

```

8 gato(orion).
9
10 ave(piolin).
11
12 dueno(ana, firulais).
13 dueno(ana, misu).
14
15 dueno(luis, luna).
16 dueno(luis, orion).
17 dueno(luis, firulais).
18
19 dueno(maria, piolin).
20
21 dueno(julia, chanel).
22
23 dueno(pedro, bruno).
24
25 mascota(Mascota) :- perro(Mascota).
26 mascota(Mascota) :- gato(Mascota).
27 mascota(Mascota) :- ave(Mascota).
28
29 todas_las_mascotas_tienen_dueno :-
30     forall(mascota(Mascota), dueno(_, Mascota)).

```

todas_las_mascotas_tienen_dueno.

false

?- todas_las_mascotas_tienen_dueno.

Examples

History

Solutions

☐ table results

Run

9.

Reto1



File

Edit

Examples

Help

119 users online

Search

Q




Program

+

```

11
12 dueno(ana, firulais).
13 dueno(ana, misu).
14
15 dueno(luis, luna).
16 dueno(luis, orion).
17 dueno(luis, firulais).
18
19 dueno(maria, piolin).
20
21 dueno(julia, chanel).
22
23 dueno(pedro, bruno).
24
25
26 amante_animales(Persona) :-
27     dueno(Persona, Mascota1),
28     perro(Mascota1),
29     dueno(Persona, Mascota2),
30     gato(Mascota2).
31 amante_animales(Personas) :-
32     findall(Persona, amante_animales(Persona), PersonasConRepetidos),
33     sort(PersonasConRepetidos, Personas).

```



amantes_animales(Personas).

Personas = [ana, luis]

?- amantes_animales(Personas).

Examples

History

Solutions

☐ table results

Run!

Reto 2



File

Edit

Examples

Help

118 users online

Search

Q



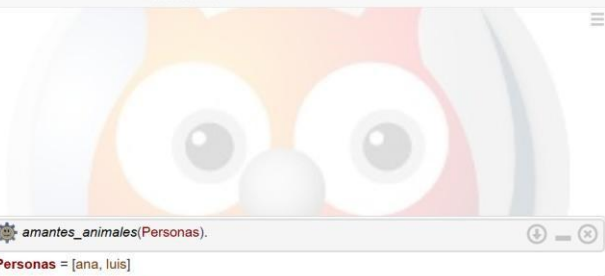

Program

+

```

10 ave(piolin).
11
12 dueno(ana, firulais).
13 dueno(ana, misu).
14
15 dueno(luis, luna).
16 dueno(luis, orion).
17 dueno(luis, firulais).
18
19 dueno(maria, piolin).
20
21 dueno(julia, chanel).
22
23 dueno(pedro, bruno).
24
25
26 amante_animales(Persona) :-
27     dueno(Persona, Mascota1),
28     perro(Mascota1),
29     dueno(Persona, Mascota2),
30     gato(Mascota2).
31 amantes_animales(Personas) :-
32     setof(Persona, amante_animales(Persona), Personas).

```



amantes_animales(Personas).

Personas = [ana, luis]

amantes_animales(Personas).

Personas = [ana, luis]

?- amantes_animales(Personas).

Examples

History

Solutions

☐ table results

Run!

The SWISH IDE interface displays a Prolog program in the left pane and its execution results in the right pane. The program defines a set of facts and a query.

```

10 ave(piolin).
11
12 dueno(ana, firulais).
13 dueno(ana, misu).
14
15 dueno(luis, luna).
16 dueno(luis, orion).
17 dueno(luis, firulais).
18
19 dueno(maria, piolin).
20
21 dueno(julia, chanel).
22
23 dueno(pedro, bruno).
24
25
26 mascota_compartida(Persona1, Persona2, Mascota) :-
27     dueno(Persona1, Mascota),
28     dueno(Persona2, Mascota),
29     Persona1 \== Persona2.
30
31 mascotas_compartidas(Persona1, Persona2, Mascotas) :-
32     setof(Mascota, mascota_compartida(Persona1, Persona2, Mascota), Mascotas).

```

The right pane shows the execution of the query `mascotas_compartidas(Persona1, Persona2, Mascotas).` with the following results:

```

Mascotas = [firulais],
Persona1 = ana,
Persona2 = luis
Mascotas = [firulais],
Persona1 = luis,
Persona2 = ana

```

At the bottom, there are tabs for Examples, History, and Solutions, and a checkbox for "table results".

Caso adicional

The SWISH IDE interface displays a modified Prolog program in the left pane and its execution results in the right pane. The program defines a set of facts and a query.

```

12 dueno(ana, firulais).
13 dueno(ana, misu).
14 dueno(ana, luna). % <-- nuevo hecho
15
16
17 dueno(luis, luna).
18 dueno(luis, orion).
19 dueno(luis, firulais).
20
21 dueno(maria, piolin).
22
23 dueno(julia, chanel).
24
25 dueno(pedro, bruno).
26
27
28 mascota_compartida(Persona1, Persona2, Mascota) :-
29     dueno(Persona1, Mascota),
30     dueno(Persona2, Mascota),
31     Persona1 \== Persona2.
32
33 mascotas_compartidas(Persona1, Persona2, Mascotas) :-
34     setof(Mascota, mascota_compartida(Persona1, Persona2, Mascota), Mascotas).

```

The right pane shows the execution of the query `mascotas_compartidas(ana, luis, Mascotas).` with the following results:

```

Mascotas = [firulais, luna]

```

At the bottom, there are tabs for Examples, History, and Solutions, and a checkbox for "table results".

8.PROBLEMA DE LA RANA

SWISH File Edit Examples Help

111 users online

Program

```

7
8 salto_maximo(4.0).
9
10
11 siguiente_estado(pos(LugarActual), pos(LugarSiguiente)) :-
12     ubicacion(LugarActual, X1, Y1),
13     ubicacion(LugarSiguiente, X2, Y2),
14     LugarActual \== LugarSiguiente,
15     salto_maximo(Max),
16     Distancia is sqrt((X2 - X1)**2 + (Y2 - Y1)**2),
17     Distancia <= Max.
18
19 dfs(pos(orilla_final), Visitados, Solucion) :-
20     reverse(Visitados, Solucion).
21
22 dfs(EstadoActual, Visitados, Solucion) :-
23     siguiente_estado(EstadoActual, EstadoSiguiente),
24     \+ member(EstadoSiguiente, Visitados),
25     dfs(EstadoSiguiente, [EstadoSiguiente | Visitados], Solucion).
26
27 buscar_solucion(Solucion) :-
28     EstadoInicial = pos(orilla_inicial),
29     dfs(EstadoInicial, [EstadoInicial], Solucion).

```

siguiente_estado(pos(orilla_inicial), pos(piedra1)).
true

siguiente_estado(pos(orilla_inicial), pos(orilla_final)).
false

buscar_solucion(Solucion).
Solucion = [pos(orilla_inicial), pos(piedra1), pos(piedra2), pos(piedra3), pos(orilla_final)]
false

buscar_solucion(_).
true

Next 10 100 1,000 Stop

buscar_solucion(Solucion).
Solucion = [pos(orilla_inicial), pos(piedra1), pos(piedra2), pos(piedra3), pos(orilla_final)]
Next 10 100 1,000 Stop

?- buscar_solucion(Solucion).

Examples History Solutions

table results Run!

9. PROBLEMA DE BATMAN VS VILLANOS

SWISH File Edit Examples Help

112 users online

Program

```

14 % --- REGLAS ---
15
16 siguiente_estado(estado(Villanos, Poderes, Energia), estado(VillanosRestantes,
17     select(villain(Nombre, Salud, Debilidades), Villanos, VillanosRestantes),
18     member(NombrePoder, Debilidades),
19     member(power(NombrePoder, Dano, Costo), Poderes),
20     Dano >= Salud,
21     Costo <= Energia,
22     EnergiaRestante is Energia - Costo.
23
24 dfs(estado([], _, _), _).
25
26 dfs(EstadoActual, Visitados) :-
27     siguiente_estado(EstadoActual, EstadoSiguiente),
28     \+ member(EstadoSiguiente, Visitados),
29     dfs(EstadoSiguiente, [EstadoSiguiente | Visitados]).
30
31 batman_can_win(EnergiaMaxima) :-
32     power_list(Superpoderes),
33     villain_list(Villanos),
34     EstadoInicial = estado(Villanos, Superpoderes, EnergiaMaxima),
35     dfs(EstadoInicial, [EstadoInicial]).

```

batman_can_win(60).
Singleton variables: [Nombre]
true

batman_can_win(59).
Singleton variables: [Nombre]
false

batman_can_win(1000).
Singleton variables: [Nombre]
true

batman_can_win(0).
Singleton variables: [Nombre]
false

?- batman_can_win(0).

Examples History Solutions

table results Run!

**1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes?
(Horas/Hombre)**

El laboratorio lo realicé en grupo con Kamilo. Entre los dos invertimos un total aproximado de 2 horas y 30 minutos, repartidas principalmente en entender el uso de findall, setof y bagof, y en la parte final resolviendo los problemas de búsqueda con DFS (la rana y Batman vs villanos), que fue lo que más tiempo nos tomó.

2. ¿Cuál es el estado actual del laboratorio? ¿Por qué?

El laboratorio quedó completo, con todos los ejercicios funcionando correctamente. Logramos resolver desde las consultas básicas con findall hasta los problemas más complejos de búsqueda en espacio de estados con DFS, verificando cada solución con las consultas de prueba correspondientes.

3. ¿Cuál considera fue el mayor logro? ¿Por qué?

Para mí el mayor logro fue entender bien la diferencia entre findall, setof y bagof, porque al principio los tres parecían hacer lo mismo, pero cuando trabajamos con listas que tenían valores repetidos (como los dueños con varias mascotas) nos dimos cuenta de que cada uno maneja los duplicados y el agrupamiento de forma distinta. Eso nos ayudó mucho a la hora de armar la lógica de los problemas de DFS más adelante.

4. ¿Cuál considera que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

El mayor problema técnico fue armar la función siguiente_estado/2 en los problemas de búsqueda, sobre todo el de Batman vs villanos, porque había que combinar varias condiciones a la vez (buscar debilidad del villano, buscar el poder correspondiente, verificar el daño y la energía disponible). Para resolverlo fuimos dividiendo el problema en partes más pequeñas, probando cada condición por separado antes de juntarlas todas en una sola regla, y usando select/3 para poder ir descontando los villanos ya derrotados de la lista.

5. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

Como equipo, Kamilo y yo nos dividimos bien las partes del laboratorio y nos explicábamos mutuamente los conceptos cuando alguno no entendía algo, sobre todo con la lógica de bagof y el uso del operador ^. Para mejorar, nos comprometemos a empezar con más anticipación los laboratorios que tienen problemas de búsqueda (como el de la rana y Batman), porque son los que más tiempo de análisis requieren antes de empezar a codificar.

6. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

Para este laboratorio no consultamos muchas fuentes externas, principalmente nos apoyamos en el material visto en clase. Lo único adicional que revisamos fue un video en YouTube para reforzar la idea de cómo funciona el backtracking en Prolog, lo cual nos ayudó a entender mejor cómo se comportan predicados como bagof/3 y el algoritmo de DFS en los problemas de búsqueda.

MyCyberAcademy. (2013). *Tutorial Prolog - 8 - Backtracking* [Video]. YouTube.

https://www.youtube.com/watch?v=KVv_t28t1QI